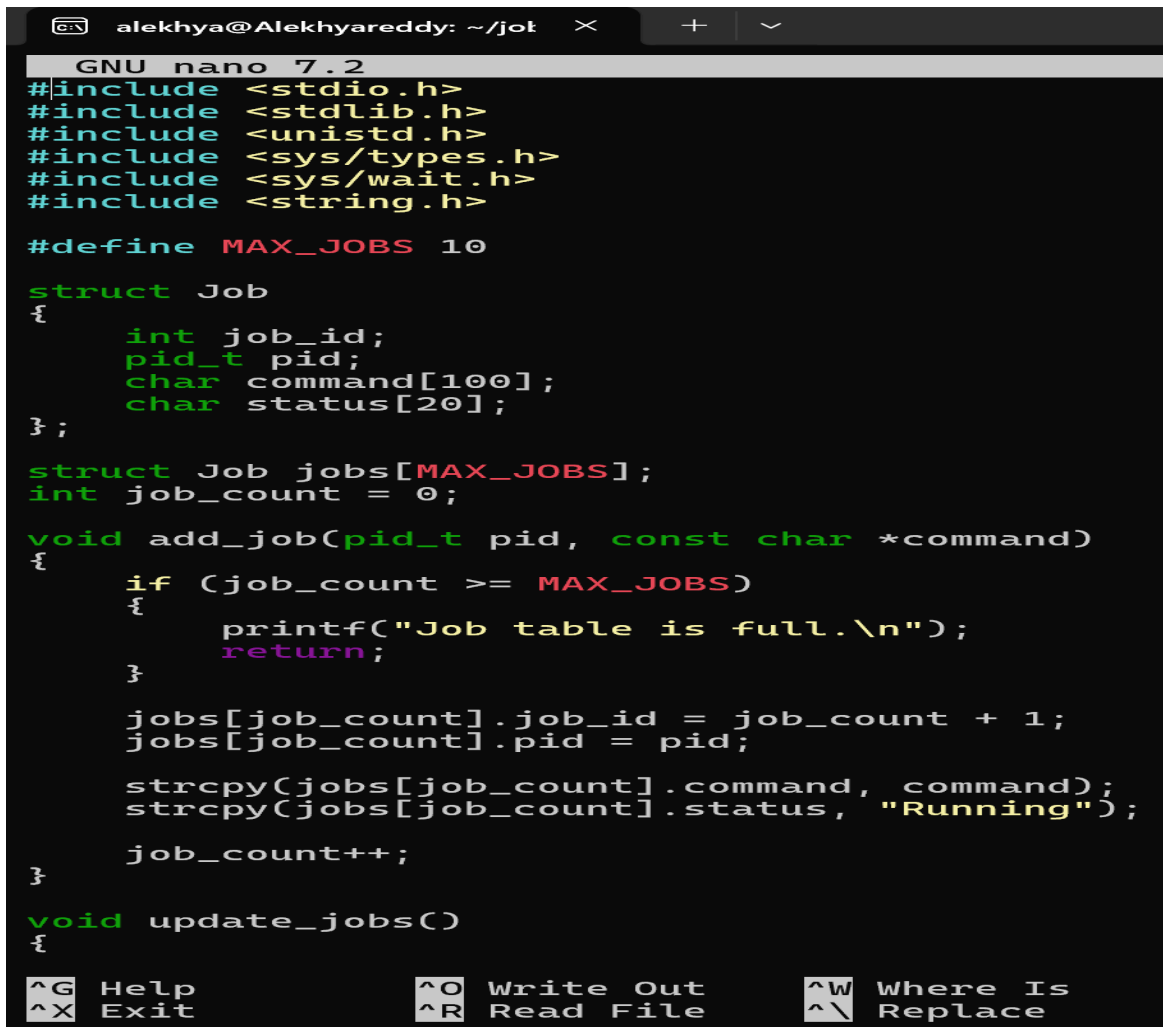


SKILL WEEK-9

1 . To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, and Test Job Listings.

Aim : To implement a simple job-management system that can create background jobs, assign job IDs, maintain their process IDs and states, list active jobs, and update job status when processes finish.

Program :



```
alekhya@Alekhyaireddy: ~/jot × + v
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10

struct Job
{
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].status, "Running");

    job_count++;
}

void update_jobs()
{
    ^G Help      ^O Write Out  ^W Where Is
    ^X Exit      ^R Read File  ^_ Replace
```

```
for (int i = 0; i < job_count; i++)
{
    if (strcmp(jobs[i].status, "Running") == 0)
    {
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );

        if (result == jobs[i].pid)
        {
            if (WIFEXITED(status))
            {
                strcpy(jobs[i].status, "Done");
            }
            else
            {
                strcpy(jobs[i].status, "Terminated");
            }
        }
    }
}

void list_jobs()
{
    update_jobs();

    printf("\n");
    printf("==== ACTIVE JOBS =====\n");

    printf("%-8s %-10s %-15s %s\n",
        "Job ID",
        "PID",
        "Status",
        "Command");
}
```

```
alekhya@Alekhyareddy: ~/jot × + v
GNU nano 7.2
        "Command");

    printf("-----\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("%-8d %-10d %-15s %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].status,
                jobs[i].command);
    }

    if (job_count == 0)
    {
        printf("No jobs found.\n");
    }

    printf("-----\n");
}

void start_job(int seconds)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return;
    }

    if (pid == 0)
    {
        printf("Child process started. PID = %d\n",
                getpid());

        sleep(seconds);

        printf("Child process %d completed.\n",
                getpid());
    }
}
```

Help **Write Out** **Where Is** **Cut**
Exit **Read File** **Replace** **Paste**

```
alekhya@Alekhyareddy: ~/jot × + v
GNU nano 7.2
printf("==== JOB MANAGEMENT SYSTEM ==== \n");

while (1)
{
    printf("\n");
    printf("1. Start Background Job\n");
    printf("2. List Jobs\n");
    printf("3. Exit\n");

    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice == 1)
    {
        printf("Enter job duration in seconds: ");
        scanf("%d", &seconds);

        start_job(seconds);
    }
    else if (choice == 2)
    {
        list_jobs();
    }
    else if (choice == 3)
    {
        update_jobs();
        printf("\nExiting job manager...\n");
        break;
    }
    else
    {
        printf("Invalid choice.\n");
    }
}

return 0;
}
```

Help **Write Out** **Where Is** **Cut**
Exit **Read File** **Replace** **Paste**

Output :

```
alekhya@Alekhyareddy: ~/jot × + ∨
alekhya@Alekhyareddy:~$ mkdir job_practical
alekhya@Alekhyareddy:~$ cd job_practical
alekhya@Alekhyareddy:~/job_practical$ pwd
/home/alekhya/job_practical
alekhya@Alekhyareddy:~/job_practical$ ls
alekhya@Alekhyareddy:~/job_practical$ nano jobs.c
alekhya@Alekhyareddy:~/job_practical$ ls
jobs.c
alekhya@Alekhyareddy:~/job_practical$ cat jobs.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10

struct Job
{
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].status, "Running");

    job_count++;
}

void update_jobs()
```



alekhya@Alekhyareddy: ~/job



```
{
    int status;

    for (int i = 0; i < job_count; i++)
    {
        if (strcmp(jobs[i].status, "Running") == 0)
        {
            pid_t result = waitpid(
                jobs[i].pid,
                &status,
                WNOHANG
            );

            if (result == jobs[i].pid)
            {
                if (WIFEXITED(status))
                {
                    strcpy(jobs[i].status, "Done");
                }
                else
                {
                    strcpy(jobs[i].status, "Terminated");
                }
            }
        }
    }
}

void list_jobs()
{
    update_jobs();

    printf("\n");
    printf("==== ACTIVE JOBS =====\n");

    printf("%-8s %-10s %-15s %s\n",
        "Job ID",
        "PID",
        "Status",
        "Command");

    printf("-----\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("%-8d %-10d %-15s %s\n",
            jobs[i].job_id,
```



alekhya@Alekhyaireddy: ~/jot



```
{
    printf("%-8d %-10d %-15s %s\n",
           jobs[i].job_id,
           jobs[i].pid,
           jobs[i].status,
           jobs[i].command);
}

if (job_count == 0)
{
    printf("No jobs found.\n");
}

printf("-----\n");
}

void start_job(int seconds)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return;
    }

    if (pid == 0)
    {
        printf("Child process started. PID = %d\n",
               getpid());

        sleep(seconds);

        printf("Child process %d completed.\n",
               getpid());

        exit(0);
    }
    else
    {
        char command[100];

        snprintf(command,
                 sizeof(command),
                 "sleep %d",
                 seconds);
    }
}
```



alekhya@Alekhyareddy: ~/jok



```
        add_job(pid, command);

        printf("Job [%d] started with PID %d\n",
               job_count,
               pid);
    }
}

int main()
{
    int choice;
    int seconds;

    printf("==== JOB MANAGEMENT SYSTEM ==== \n");

    while (1)
    {
        printf("\n");
        printf("1. Start Background Job\n");
        printf("2. List Jobs\n");
        printf("3. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        if (choice == 1)
        {
            printf("Enter job duration in seconds: ");
            scanf("%d", &seconds);

            start_job(seconds);
        }
        else if (choice == 2)
        {
            list_jobs();
        }
        else if (choice == 3)
        {
            update_jobs();

            printf("\nExiting job manager...\n");
            break;
        }
        else
        {
            printf("Invalid choice.\n");
        }
    }
}
```

alekhya@Alekhyareddy: ~/job

```
}  
  
    return 0;  
}  
alekhya@Alekhyareddy:~/job_practical$ gcc jobs.c -o jobs  
alekhya@Alekhyareddy:~/job_practical$ ls  
jobs  jobs.c  
alekhya@Alekhyareddy:~/job_practical$ ./jobs  
===== JOB MANAGEMENT SYSTEM =====
```

```
1. Start Background Job  
2. List Jobs  
3. Exit  
Enter choice: 1  
Enter job duration in seconds: 5  
Job [1] started with PID 573
```

```
1. Start Background Job  
Child process started. PID = 573  
2. List Jobs  
3. Exit  
Enter choice: Child process 573 completed.  
2
```

```
===== ACTIVE JOBS =====  
Job ID    PID      Status      Command  
-----  
1         573      Done        sleep 5  
-----
```

```
1. Start Background Job  
2. List Jobs  
3. Exit  
Enter choice: 2
```

```
===== ACTIVE JOBS =====  
Job ID    PID      Status      Command  
-----  
1         573      Done        sleep 5  
-----
```

```
1. Start Background Job  
2. List Jobs  
3. Exit  
Enter choice: 1  
Enter job duration in seconds: 10  
Job [2] started with PID 580
```

1. Start Background Job
2. List Jobs
3. Exit

Enter choice: Child process started. PID = 580

2

===== ACTIVE JOBS =====

Job ID	PID	Status	Command
1	573	Done	sleep 5
2	580	Running	sleep 10

1. Start Background Job
2. List Jobs
3. Exit

Enter choice: Child process 580 completed.

2

===== ACTIVE JOBS =====

Job ID	PID	Status	Command
1	573	Done	sleep 5
2	580	Done	sleep 10

1. Start Background Job
2. List Jobs
3. Exit

Enter choice: 9

Invalid choice.

1. Start Background Job
2. List Jobs
3. Exit

Enter choice: 3

Exiting job manager...

alekhya@Alekhyareddy:~/job_practical\$ ls -l

total 24

-rwxr-xr-x 1 alekhya alekhya 16704 Sep 19 08:45 jobs

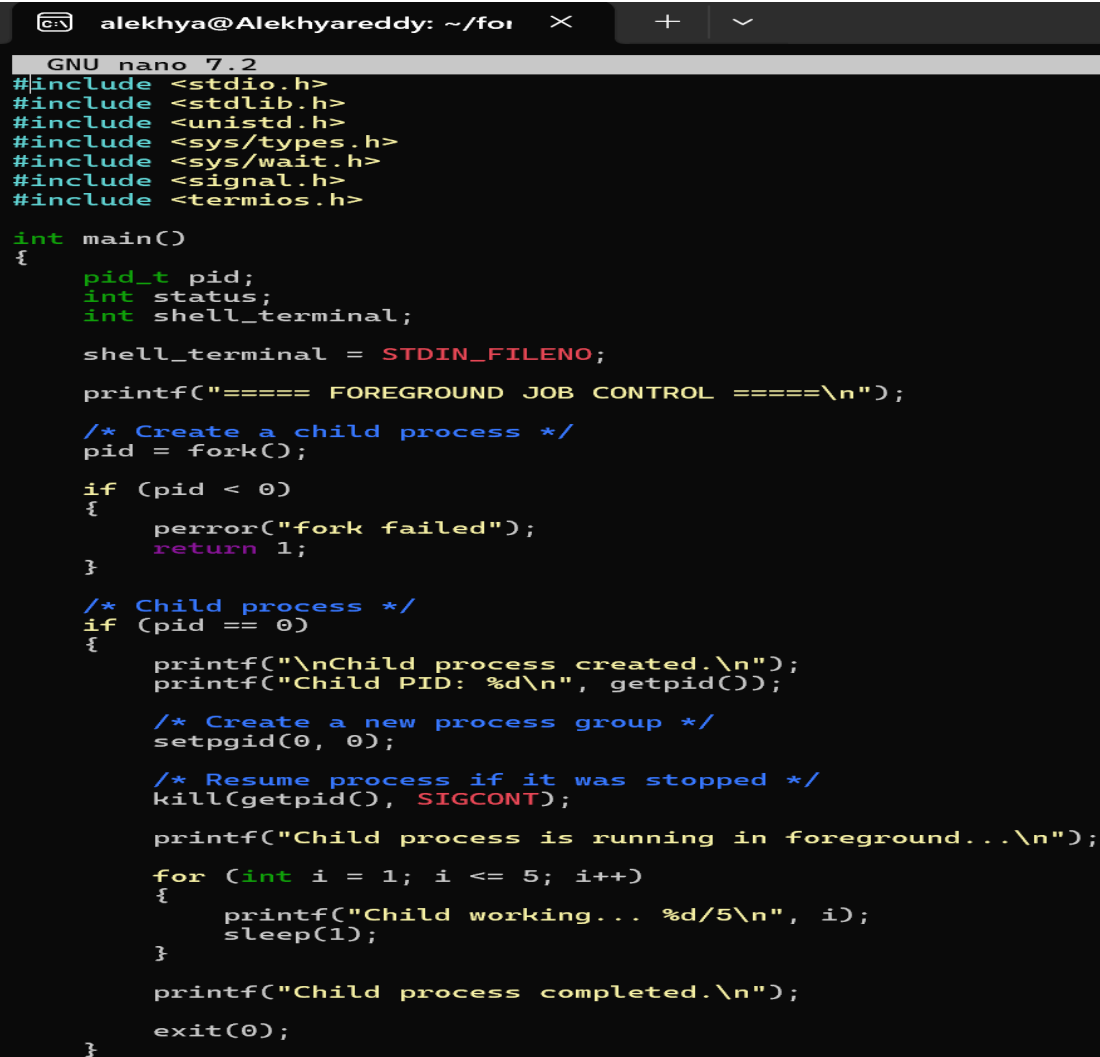
-rw-r--r-- 1 alekhya alekhya 3321 Sep 19 08:45 jobs.c

alekhya@Alekhyareddy:~/job_practical\$ |

2 . To Design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, and Test Foreground Switching.

Aim : To implement foreground job control by identifying a target job, transferring terminal control to the selected process, resuming a stopped process, waiting for its completion, updating the job state, and testing foreground switching.

Program :



```
alekhya@Alekhyareddy: ~/foi
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <termios.h>

int main()
{
    pid_t pid;
    int status;
    int shell_terminal;

    shell_terminal = STDIN_FILENO;

    printf("==== FOREGROUND JOB CONTROL ====\\n");

    /* Create a child process */
    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    /* Child process */
    if (pid == 0)
    {
        printf("\\nChild process created.\\n");
        printf("Child PID: %d\\n", getpid());

        /* Create a new process group */
        setpgid(0, 0);

        /* Resume process if it was stopped */
        kill(getpid(), SIGCONT);

        printf("Child process is running in foreground...\\n");

        for (int i = 1; i <= 5; i++)
        {
            printf("Child working... %d/5\\n", i);
            sleep(1);
        }

        printf("Child process completed.\\n");
        exit(0);
    }
}
```



alekhya@AlekhyaReddy: ~/foi



GNU nano 7.2

```
/* Parent process */

printf("\nParent process created child.\n");
printf("Child PID: %d\n", pid);

/* Put child in its own process group */
setpgid(pid, pid);

printf("\nIdentifying target job...\n");
printf("Target Job PID: %d\n", pid);

printf("\nTransferring terminal control to child...\n");

/*
 * Give terminal control to child process group.
 */
if (tcsetpgrp(shell_terminal, pid) == -1)
{
    perror("tcsetpgrp");
}

printf("Terminal control transferred.\n");

/* Wait for child */
printf("\nParent is waiting for foreground job...\n");

if (waitpid(pid, &status, WUNTRACED) == -1)
{
    perror("waitpid failed");
    return 1;
}

/* Check job state */
if (WIFSTOPPED(status))
{
    printf("\nJob stopped.\n");
    printf("Resuming foreground job...\n");
    kill(pid, SIGCONT);
    waitpid(pid, &status, 0);
}

if (WIFEXITED(status))
{
    printf("\nForeground job completed normally.\n");
    printf("Exit status: %d\n",
           WEXITSTATUS(status));
}

}

/* Give terminal back to parent */
printf("\nReturning terminal control to parent...\n");

if (tcsetpgrp(shell_terminal, getpgrp()) == -1)
{
    perror("tcsetpgrp");
}

printf("Terminal control returned to parent.\n");

printf("\nJob state updated: COMPLETED\n");

printf("\n==== FOREGROUND SWITCHING TEST COMPLETED =====\n");

return 0;
}
```

Output :

```
alekhya@AlekhyaReddy: ~/for X + v
alekhya@AlekhyaReddy:~$ mkdir foreground_job
alekhya@AlekhyaReddy:~$ cd foreground_job
alekhya@AlekhyaReddy:~/foreground_job$ pwd
/home/alekhya/foreground_job
alekhya@AlekhyaReddy:~/foreground_job$ ls
alekhya@AlekhyaReddy:~/foreground_job$ nano foreground_job.c
alekhya@AlekhyaReddy:~/foreground_job$ gcc foreground_job.c -o foreground_job
alekhya@AlekhyaReddy:~/foreground_job$ ls
foreground_job  foreground_job.c
alekhya@AlekhyaReddy:~/foreground_job$ ./foreground_job
===== FOREGROUND JOB CONTROL =====

Parent process created child.

Child process created.
Child PID: 761
Child PID: 761

Identifying target job...
Target Job PID: 761

Transferring terminal control to child...
Terminal control transferred.

Child process is running in foreground...
Parent is waiting for foreground job...
Child working... 1/5
Child working... 2/5
Child working... 3/5
Child working... 4/5
Child working... 5/5
Child process completed.

Foreground job completed normally.
Exit status: 0

Returning terminal control to parent...

[1]+  Stopped                  ./foreground_job
alekhya@AlekhyaReddy:~/foreground_job$ ls -l
total 24
-rwxr-xr-x 1 alekhya alekhya 16496 Sep 19 09:14 foreground_job
-rw-r--r-- 1 alekhya alekhya 2486 Sep 19 09:13 foreground_job.c
alekhya@AlekhyaReddy:~/foreground_job$ file foreground_job
foreground_job: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=9d3d5b6373ed02da75ae7fa27f40b4d2f18da76, for GNU/Linux 3.2.0, not stripped
alekhya@AlekhyaReddy:~/foreground_job$
```

